definition, so the range of values that the optimizer can operate over is limited and small. Sometimes probabilities may be extremely tiny or very close to 1, and this runs into numerical issues when optimizing on a computer with limited numerical precision. If we instead use a surrogate objective, namely $-\log\pi_s(a|\theta)$ (where log is the natural logarithm), we have an objective that has a larger "dynamic range" than raw probability space, since the log of probability space ranges from $(-\infty,0)$, and this makes the log probability easier to compute. Moreover, logarithms have the nice property that $\log(a \times b) = \log(a) + \log(b)$, which means when we multiply log probabilities, we can turn this multiplication into a sum, which is also more numerically stable than multiplication. If we set our objective as $-\log\pi_s(a|\theta)$ instead of $1 - \pi_s(a|\theta)$, our loss still abides by the intuition that the loss function approaches 0 as $\pi_s(a|\theta)$ approaches 1. Our gradients will be tuned to try to increase $\pi_s(a|\theta)$ to 1, where a = action 3, for our running example.

4.2.4. Credit assignment

Our objective function is $-\log\pi_s(a|\theta)$, but this assigns equal weight to every action in our episode. The weights in the network that produced the last action will be updated to the same degree as the first action. Why shouldn't that be the case? Well, it makes sense that the last action right before the reward deserves more credit for winning the game than does the first action in the episode. For all we know, the first action was actually sub-optimal, but then we later made a comeback and hit the goal. In other words, our confidence in how "good" each action is diminishes the further we are from the point of reward. In a game of chess, we attribute more credit to the last move made than the first one. We're very confident that the move that directly led to us to winning was a good move, but we become less confident the further back we go. How much did the move five time steps ago contribute to winning? We're not so sure. This is the problem of *credit assignment*.

We express this uncertainty by multiplying the magnitude of the update by the discount factor, which you learned in chapter 3 ranges from 0 to 1. The action right before the episode ends will have a discount factor of 1,